KIO Router AI — Detailed Project Progress Report

This report explains what we have accomplished so far in the project, how the system is organized, what we implemented, what we tested, and what remains to be done.

1. Project objective and scope

Our project, **KIO Router AI**, is designed to automatically classify incoming customer messages and route them to the appropriate department based on the customer's intent.

The system has two main categories:

- **Support — Flag 0:** Existing-service problems, complaints, billing questions, account assistance, and issues with existing orders or services.
- **Sales — Flag 1:** New purchases, package upgrades, pricing inquiries, promotions, and requests for offers or quotations.

The project explores a **Decision AI** approach and was originally planned to include a traditional **Machine Learning** approach for comparison. We have now cleaned the `System` branch to focus on the Decision AI implementation, while preserving the directory structure for Student 3 to add the ML component later.

The overall objective is to build an understandable, testable customer-message routing system and evaluate its behavior using a fixed dataset.

2. Dataset preparation and validation

One of our first major tasks was preparing a small, controlled dataset for the project.

- **Total messages:** 60
- **Dataset columns:** 7
- **Support examples:** 30
- **Sales examples:** 30

The dataset is **frozen**: the professor's constraint is to keep it at 60 records.

The dataset is stored at:

`data/Agent_KIO_Dataset_FROZEN.xlsx`

We established a fixed training/test arrangement:

Split	Total records	Support	Sales
Training set	40	20	20
Test set	20	10	10
Total	60	30	30

The split was designed to maintain balanced classes and avoid leakage between the training and test sets through the grouping strategy.

We also established that **mixed-intent messages should prioritize Support**. For example, if a customer reports a problem with an existing package and also asks about upgrading, the system should handle the existing problem first.

What we accomplished with the dataset

- Fixed the dataset at 60 records without adding new rows.
- Maintained an equal number of Support and Sales examples.
- Established a reproducible training/test arrangement.
- Validated the dataset's structure and integrity.
- Preserved the frozen Excel file in the repository.

The dataset's integrity checks passed. However, maintaining a clean dataset does not by itself prove that a routing system generalizes well to unseen real-world messages.

3. Decision AI implementation

The next major part was implementing the Decision AI router, which is the main component currently retained in the `System` branch.

Instead of relying on a locally trained ML classifier, the current system uses written routing instructions with the **Cloudflare Clef decision-model API**.

How the routing process works

1. **Customer message**

The user enters a message in the website.

2. **Input validation**

The system checks the input before processing.

3. **Decision AI router**

Clef evaluates the message using the project's routing instructions.

- **Support** → Flag 0
- **Sales** → Flag 1

The frontend displays the result and can demonstrate routing to the corresponding simulated agent.

Routing rules we defined

The instructions tell the decision model to distinguish between existing-service needs and new-purchase intent.

- Existing service problems, complaints, and account-related questions go to **Support**.
- New purchases, upgrades, prices, and promotions go to **Sales**.
- Mixed messages prioritize **Support**.
- The model should follow the same rules consistently instead of classifying messages based only on isolated keywords.

This distinction is important because a message can mention the word “package” in either a support context or a sales context. The customer's actual intent matters.

Clef integration

We integrated the Clef router into the shared backend. The project includes:

- `kio/clef_router.py` for the Clef routing workflow
- `kio/routing_instructions.md` for the routing instructions

The application uses environment-based configuration for credentials. The actual `.env` file is intended to remain local and excluded from Git.

We also retained a **Mock Mode** for simpler local testing. Mock Mode uses a basic heuristic and may misclassify messages with nuanced intent. It is not equivalent to the real Clef decision model.

4. Frontend development

We developed a website using **Streamlit**, a Python framework for building interactive web applications.

The main frontend file is:

`frontend/app.py`

The frontend is the user-facing layer of the project. It lets someone try the routing system without directly running individual Python functions.

Frontend features

The current implementation is designed to provide:

- A text input for customer messages.
- A way to submit messages for routing.
- A displayed routing result.
- Mock Mode for simplified testing.
- Session history for previously processed messages.
- Example messages for quick testing.
- Demonstrations of routing through simulated Support and Sales agents.
- Input validation and error handling.

We also fixed an example-button callback issue so that selecting an example can populate the intended message correctly.

What the frontend does not do

The website is a demonstration and testing interface. The simulated agents are not real employees, and the system is not connected to a real telecom customer-service platform.

It does not establish that real customer tickets are created, that live company databases are updated, or that real staff receive notifications.

These distinctions keep the project description accurate and prevent us from claiming integrations that have not been implemented.

5. Backend structure and responsibilities

We organized the application into separate modules under `kio/`, rather than putting all the logic in the frontend.

File	Responsibility
`kio/config.py`	Application configuration.
`kio/data.py`	Dataset loading and integrity checks.
`kio/base.py`	Shared routing-related functionality.
`kio/validation.py`	Input and data validation.
`kio/agents.py`	Simulated Support and Sales agents.
`kio/router.py`	Routing workflow.
`kio/clef_router.py`	Clef decision-model integration.
`kio/routing_instructions.md`	Rules used to guide message classification.
`kio/__init__.py`	Package initialization.

This modular structure makes the project easier to understand, test, maintain, and extend.

For example, the frontend handles user interaction, while the router handles the routing workflow. This separation also makes it easier for Student 3 to develop the ML component separately and integrate it later.

6. Testing and verification completed so far

We performed several checks during development and cleanup.

Check	Status based on our previous work
Dataset structure and integrity	Passed

Check	Status based on our previous work
Dataset remains frozen	Verified
Python syntax checks after cleanup	Passed
Backend and frontend imports	Passed
Example-button callback fix	Checked
Earlier Clef integration test	Successful
Frontend launch after the latest cleanup	Still needs to be retested
Real Clef test after the latest cleanup	Still needs to be rerun
Full evaluation after the latest cleanup	Still needs to be rerun

Earlier Clef tests

Before the latest cleanup, we successfully tested real Clef routing. One example asked about upgrading an existing internet package and available offers. The result was **Sales (Flag 1)**, with a reported confidence of `0.962`.

Another earlier smoke test returned **HTTP 200** and a **Support** prediction with a reported confidence of `0.9843`.

These are useful examples of successful earlier tests, but they are not a substitute for retesting the cleaned branch.

What these tests prove

The checks support that the components were syntactically valid, the project could import its modules, the dataset passed its integrity checks, and the Clef integration worked in earlier testing.

They do not prove that the current cleaned frontend launches successfully, that the API still works with the current configuration, or that the system will classify every new message correctly. Those checks remain part of the next testing stage.

7. GitHub repository cleanup

One of our latest tasks was cleaning the `System` branch of the GitHub repository:

Open KIO-Router-AI — System branch

The goal was to make this branch focus on the Decision AI system and remove visible ML implementation code, while preserving the folders Student 3 will need.

Files and components removed

According to the cleanup report, we removed:

- `kio/ml_router.py` — the old ML routing implementation.
- ML training and evaluation scripts.
- The comparison script.
- The data exploration notebook.
- Archived ML artifacts, including a saved model and supporting files.
- Previously generated evaluation result files.
- Unnecessary Python cache folders.

We also updated the frontend and configuration to remove the ML and comparison modes, leaving the Decision AI/Clef workflow and Mock Mode.

Folders preserved for Student 3

We kept the following directories with `.gitkeep` placeholders:

```
ml_model/
models/
comparison/
results/
```

These placeholders allow Git to preserve empty directories. Student 3 can add the future ML implementation, saved model artifacts, comparison scripts, and results without requiring us to redesign the repository.

Documentation and dependencies

We updated the project's documentation and dependency list to reflect the current scope. We also retained `decision_model/`, the frozen dataset, and the supporting backend modules.

The cleanup report says the dataset hash and integrity checks passed, and the code syntax and imports were checked. However, the frontend was not launched after the latest cleanup.

One important distinction: removing ML files from the current branch does not necessarily remove those files from the repository's previous Git history. The cleanup was intended to simplify the current branch, not erase its historical commits.

8. Current repository structure

Based on the latest cleanup report, the intended structure is:

```
System/
■■■ README.md
■■■ STRUCTURE.md
■■■ requirements.txt
■■■ .gitignore
■■■ .env.example
■
■■■ data/
■   ■■■ Agent_KIO_Dataset_FROZEN.xlsx
■
■■■ frontend/
■   ■■■ app.py
■   ■■■ README.md
```

```
■
■■■ kio/
■   ■■■ __init__.py
■   ■■■ config.py
■   ■■■ data.py
■   ■■■ base.py
■   ■■■ validation.py
■   ■■■ agents.py
■   ■■■ router.py
■   ■■■ clef_router.py
■   ■■■ routing_instructions.md
■
■■■ decision_model/
■   ■■■ README.md
■   ■■■ smoke_test_clef.py
■   ■■■ evaluate_decision_model.py
■
■■■ ml_model/
■   ■■■ .gitkeep
■■■ models/
■   ■■■ .gitkeep
■■■ comparison/
■   ■■■ .gitkeep
■■■ results/
    ■■■ .gitkeep
```

The actual GitHub branch should be checked before treating this as a guaranteed, up-to-date file listing.

9. Project responsibilities and future integration

The project was originally organized around three students with different responsibilities.

- **Student 1 — Project foundation:** Problem definition, architecture, dataset preparation, and data integrity.
- **Student 2 — Decision AI:** Routing instructions, Clef integration, frontend/backend workflow, and Decision AI evaluation.
- **Student 3 — Machine Learning:** ML implementation, training and evaluation, and comparison with the Decision AI approach.

The current cleanup supports this separation: the `System` branch contains the Decision AI work, while the ML and comparison folders remain available for Student 3.

When the ML component is ready, it can be developed and integrated without unnecessarily mixing its code with the current Clef router.

10. Important limitations we have identified

There are several things we should be transparent about when presenting the project.

1. **Small dataset:** The dataset has only 60 records. Even strong results on 20 test examples would not establish broad real-world reliability.
2. **No current ML implementation in this branch:** The ML component is reserved for Student 3.
3. **Mock Mode is simplified:** It uses basic heuristics and may fail on nuanced customer messages.

4. **External API dependency:** Real Clef mode requires valid credentials, network connectivity, and API availability.
5. **Simulated agents:** The project demonstrates routing rather than performing real telecom operations.
6. **Frontend retesting required:** We need to launch and test the cleaned application again.
7. **Dataset publishing permission:** The dataset's license and permission for public redistribution have not yet been verified. This must be resolved before deciding to publish it publicly.

11. What remains to be done?

Here is the practical next-stage checklist.

Next steps

- ■ Run the cleaned frontend
- ■ Test Mock Mode
- ■ Test real Clef mode
- ■ Run the decision evaluation
- ■ Review GitHub changes
- ■ Verify dataset permission
- ■ Integrate Student 3's ML work later

Details

- **Run the cleaned frontend:** Launch the app from the actual cloned GitHub repository.
- **Test Mock Mode:** Submit one Support message and one Sales message.
- **Test real Clef mode:** Verify both categories with Mock Mode disabled and valid local credentials.
- **Run the decision evaluation:** Check the evaluation script and document its metrics and limitations.
- **Review GitHub changes:** Confirm the intended files are present and `.env` and credentials are not committed.
- **Verify dataset permission:** Check whether the frozen dataset can legally be redistributed publicly.
- **Integrate Student 3's ML work later:** Add the ML implementation and comparison only when ready.

12. Overall status

At this point, we have established the dataset, defined the Support/Sales routing rules, built the Decision AI/Clef workflow and Streamlit frontend, organized the backend into modules, performed initial tests, and cleaned the `System` branch to exclude the ML implementation while preserving the folders needed for future work.

The project is now positioned for final verification of the cleaned Decision AI application, followed by completing the evaluation and coordinating the future ML integration.

The most important thing is to distinguish what we have implemented and checked from what still needs verification. We have a structured project and an earlier successful Clef integration test, but we should retest

the cleaned application before declaring the current branch fully verified.